

M7_Practice

Danni Martin

2024-12-09

Time Series IV - ARIMA Model

This module introduces the ARIMA model. ARIMA models are suitable for analyzing and forecasting time series data with constant mean and variance over time (stationary). By capturing the autocorrelation structure in the data, these models are particularly helpful in understanding the dynamics and relationships within a time series.

SES models based on trend and seasonality of data, while ARIMA describes autocorrelations in data.

Time series with trend or seasonality are NOT stationary.

Time series with cyclic behavior ARE stationary.

Preliminaries

Key Words/Concepts:

- White Noise: sequence of random data points with no correlation.
- Stationary: no trend and homoskedastic
- Differencing: Compute differences between consecutive observations. Make non-stationary series stationary.
- Homoskedasticity: constant/uniform variance
- Heteroskedasticity: increasing variance
- Box-Cox Transformation: Used to normalize variance.
- Unit Root Tests: tests to determine if series is stationary.
- Autocorrelation: summarizes relationships between a series and its lags.
- Partial Autocorrelation: Similar to ACF but intervening lags are removed.

Models:

- AR: Autoregressive. Forecast using linear combination of past values of the variable.
- MA: Moving Averages. Forecast using past errors.
- ARIMA: Autoregressive Integrated Moving Averages. Combines all three by considering its own past, removing trends, and accounting for past errors.

White Noise

White noise: sequence of random data points with no correlation.

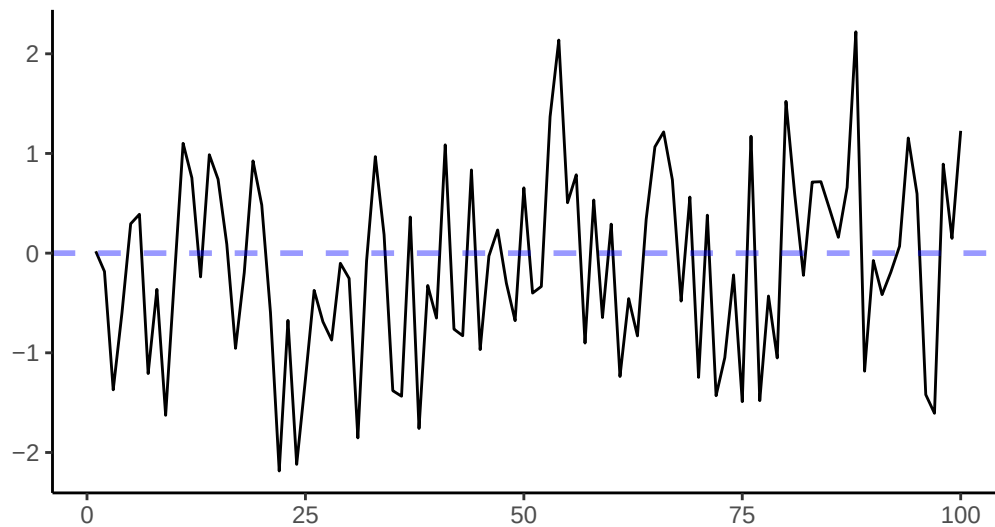
```
# Generate white noise process.
# Simulate data - normal distribution with a mean of zero and constant variance.

library(fpp3)
set.seed(10)
wn<-tsibble(x=rnorm(100),period=seq(1:100),index=period)

# Plot.
# Series behaves erratically but fluctuates around mean of 0 with sd of 1.
wn %>% autoplot(x) + theme_classic() +
  labs(title="White Noise Process",
        subtitle="Mean=0 and Standard Deviation=1",
        x="",y="") +
  geom_hline(yintercept = 0, col="blue", lwd=1, linetype="dashed",
            alpha=0.4)
```

White Noise Process

Mean=0 and Standard Deviation=1



Stationary & Differencing

Time series are stationary when statistical properties do not change over time, ie constant mean, variance, and auto-covariance. White noise process is an example of a stationary process.

Stationary: No trend and is homoskedastic.

Many time models, including ARIMA, require a stationary time series. ARIMA models do not work well if the statistical properties change over time.

```
# Load dataset
tesla<-tsibble(
  period=yearquarter(c("2016:Q1", "2016:Q2", "2016:Q3", "2016:Q4",
    "2017:Q1", "2017:Q2", "2017:Q3", "2017:Q4",
    "2018:Q1", "2018:Q2", "2018:Q3", "2018:Q4",
    "2019:Q1", "2019:Q2", "2019:Q3", "2019:Q4",
    "2020:Q1", "2020:Q2", "2020:Q3", "2020:Q4",
    "2021:Q1", "2021:Q2", "2021:Q3", "2021:Q4",
    "2022:Q1", "2022:Q2", "2022:Q3", "2022:Q4",
    "2023:Q1", "2023:Q2")),
  deliveries=c(14.8, 14.4, 24.5, 22.2,
    25, 22, 26.2, 29.9,
```

```

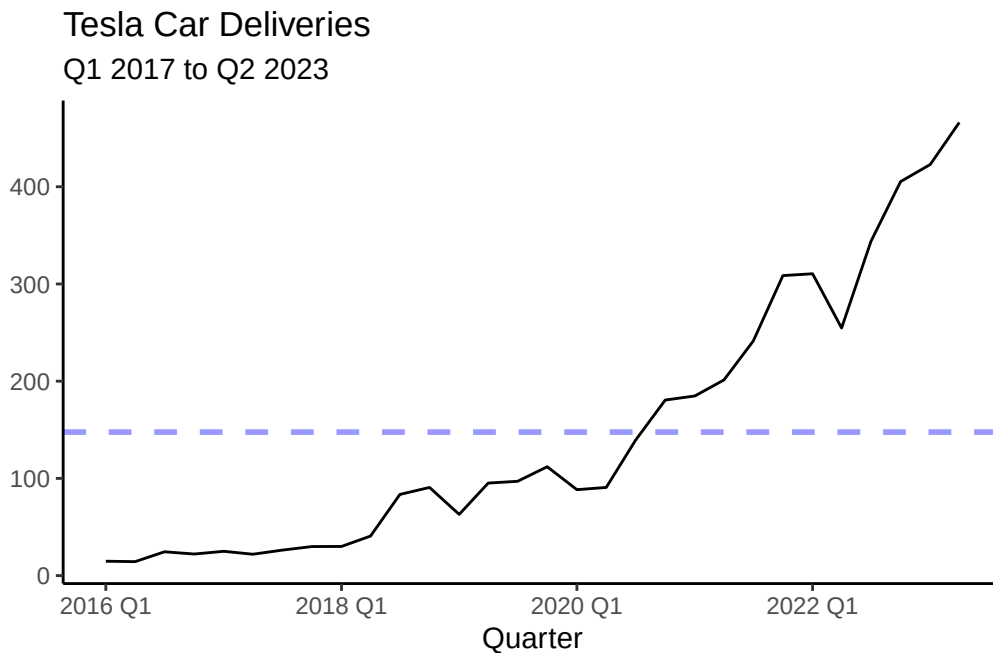
30,40.7,83.5,90.7,
63,95.2,97,112,
88.4,90.7,139.3,180.6,
184.82,201.25,241.3,308.6,
310.5,254.7,343.8,405.3,
422.9,466.1),
index=period      # This is the time variable
)

```

```

# Plot tesla deliveries.
# Upward trend; therefore, series is NOT stationary.
tesla %>% autoplot(.vars=deliveries) + theme_classic() +
  labs(title= "Tesla Car Deliveries",
        subtitle = "Q1 2017 to Q2 2023") +
  xlab("Quarter") + ylab(" ") +
  geom_hline(yintercept = mean(tesla$deliveries), col="blue", linetype="dashed", lwd=1, alpha=0.5)

```



Differencing: compute differences in between consecutive observations. Used to make non-stationary series stationary.

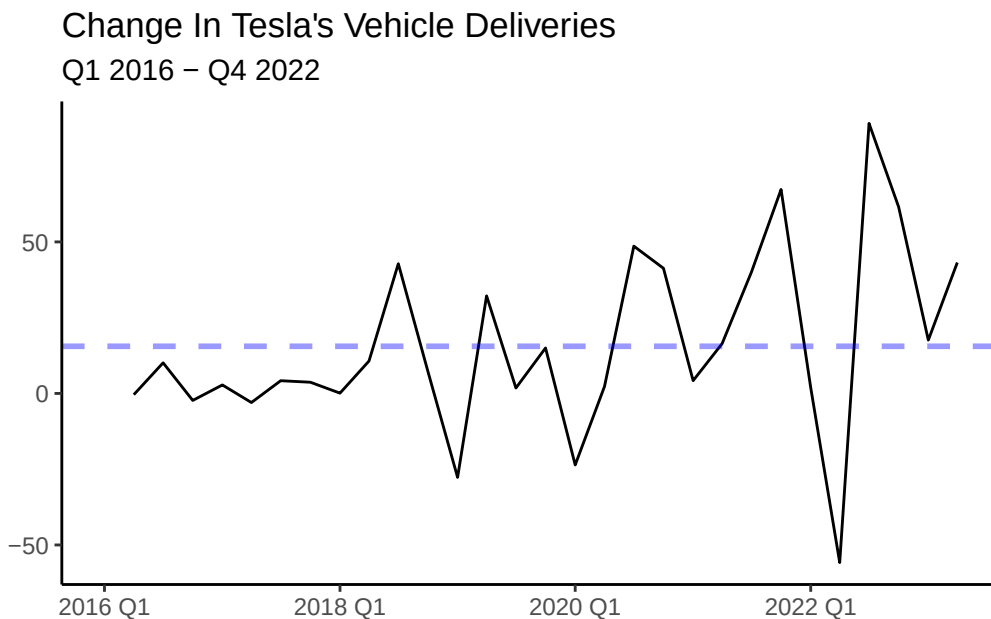
```

# First difference.
# Eliminate trend using the difference() function. See plot.

```

```
# Seems to be some correlation between quarters.
tesla %>%
  as_tsibble(index=period, regular=T) %>% autoplot(difference(deliveries)) + theme_classic()
  labs(title="Change In Tesla's Vehicle Deliveries",
        subtitle = "Q1 2016 - Q4 2022",x="",y="") +
  geom_hline(yintercept = mean(difference(tesla$deliveries), na.rm = TRUE), col="blue", line
```

Warning: Removed 1 row containing missing values or values outside the scale range (`geom\_line()`).



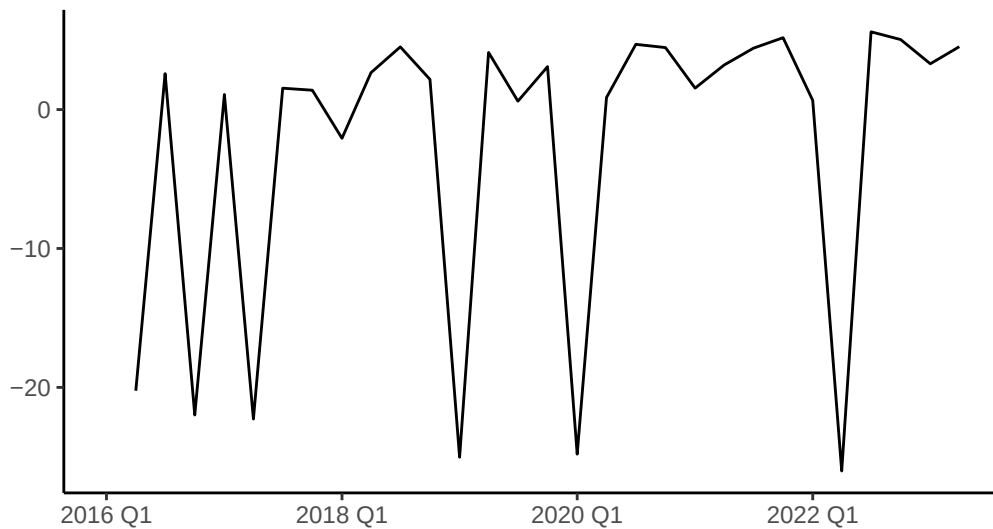
```
# Normalize variance in the series using a Box-Cox transformation.
lambda <- tesla %>% as_tsibble(index=period, regular=T) %>%
  features(deliveries, features = guerrero) %>% pull(lambda_guerrero)

tesla %>% as_tsibble(index=period, regular=T) %>%
  autoplot(box_cox(difference(deliveries), lambda)) +
  labs(y = "") + theme_classic() +
  labs(title="Box-Cox Transformation of Tesla's Vehicle Deliveries",
        subtitle = "Q1 2016 - Q4 2022",x="",y="")
```

Warning: Removed 1 row containing missing values or values outside the scale range (`geom\_line()`).

Box-Cox Transformation of Tesla's Vehicle Deliveries

Q1 2016 – Q4 2022



Unit Root Tests

Kwiatkowski-Phillips-Schmidt-Shin (KPSS) test In this test, null hypothesis is that data are stationary, looking for evidence that the null hypothesis is false. Small p values suggest null hypothesis is false; therefore, differencing is required.

ndiffs determines number of differences required to make data stationary.

```
tesla %>% as_tsibble(index=period, regular=T) %>%  
  features(deliveries, features = c(unitroot_kpss, unitroot_ndiffs))
```

```
# A tibble: 1 x 3  
  kpss_stat kpss_pvalue ndiffs  
    <dbl>      <dbl>   <int>  
1    0.996      0.01     2
```

Autocorrelation

```

# Income growth in California.
# Load data and create test/train sets.
library(fpp3)
library(tidyverse)

-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v forcats 1.0.0      v readr    2.1.5
v purrr   1.0.2      v stringr 1.5.1
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter()      masks stats::filter()
x tsibble::interval() masks lubridate::interval()
x dplyr::lag()          masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
errors.

PI<-read_csv("https://jagelves.github.io/Data/PersonalIncome.csv")

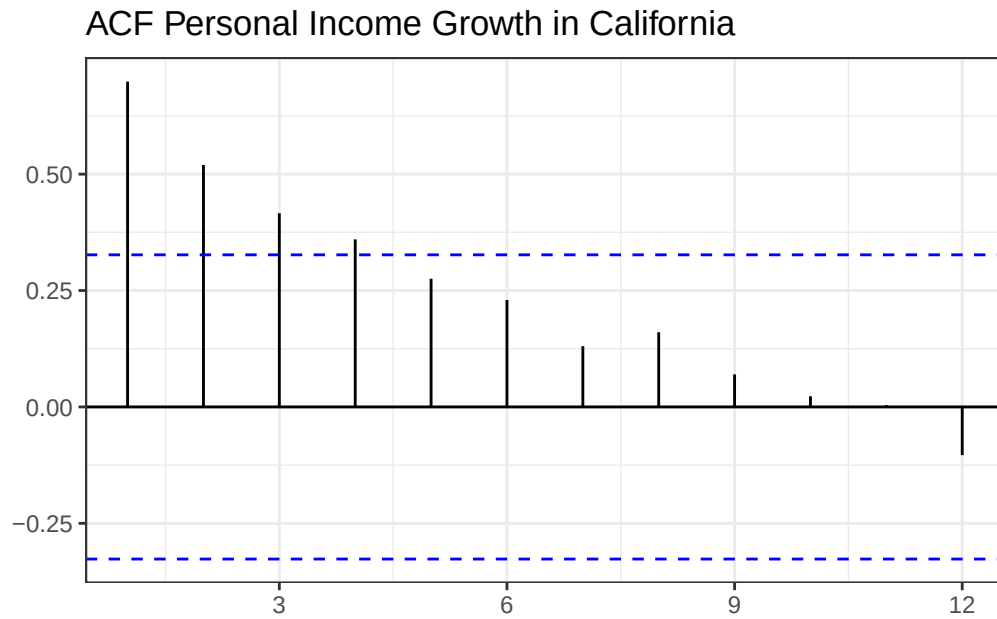
Rows: 92 Columns: 2
-- Column specification -----
Delimiter: ","
dbl (2): Date, PI_Growth

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.

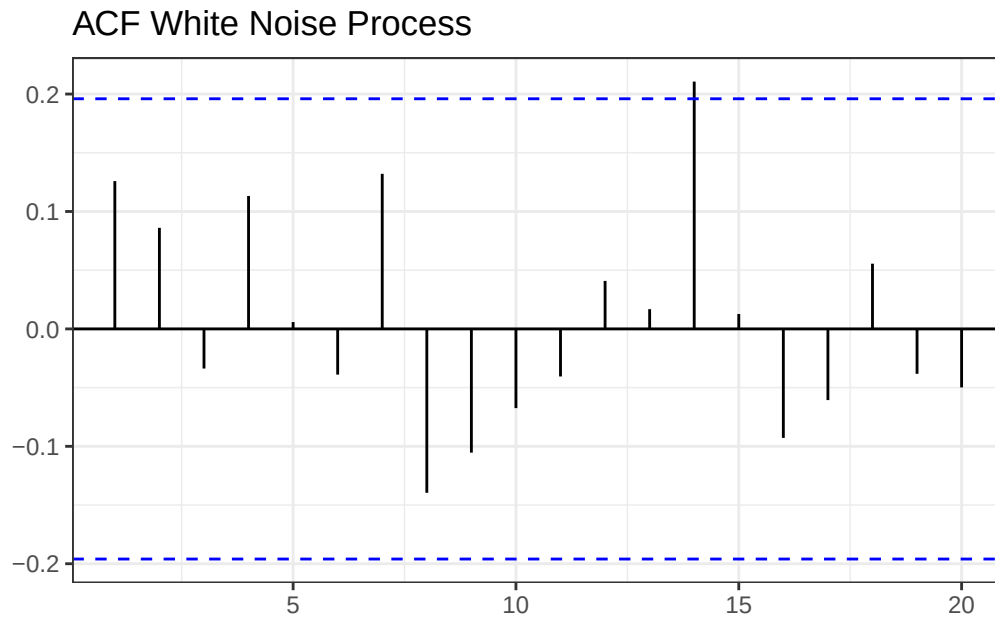
PI %>% as_tsibble(index=Date) %>%
  filter_index(1970~2005) -> PI_train
PI %>% as_tsibble(index=Date) %>%
  filter_index(2006~2021) -> PI_test

# Apply and plot ACF function. Blue line indicates which ACs are statistically significant.
# Correlation decreases as lag increases.
PI_train %>%
  ACF(lag_max = 12,PI_Growth) %>%
  autoplot() + theme_bw() + labs(x="", y="",
                                title="ACF Personal Income Growth in California")

```



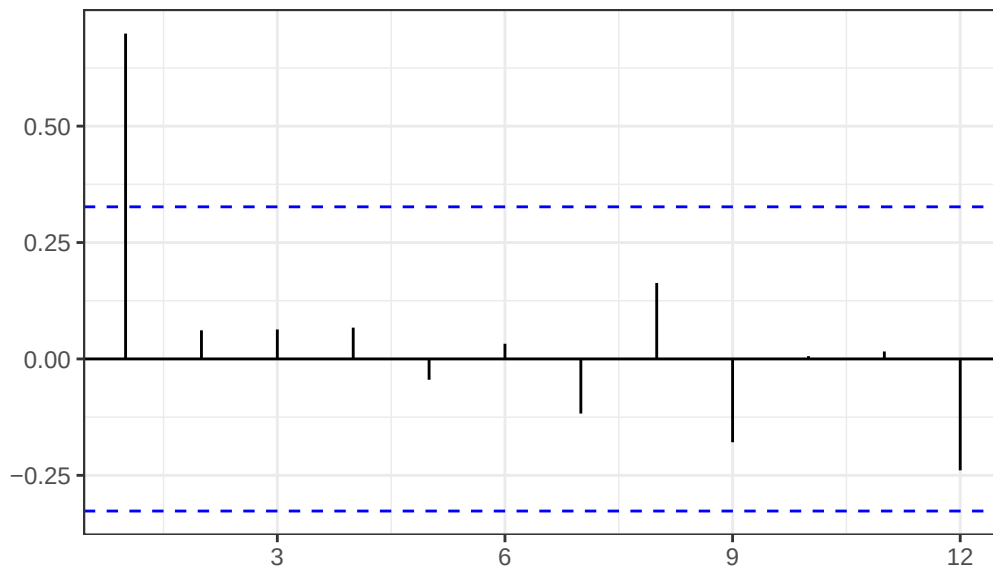
```
# Apply ACF to white noise data.  
# No correlation.  
wn %>% ACF(x) %>% autoplot() + theme_bw() + labs(x="", y="ACF") +  
labs(x="", y="", title="ACF White Noise Process")
```



Partial Autocorrelation Function

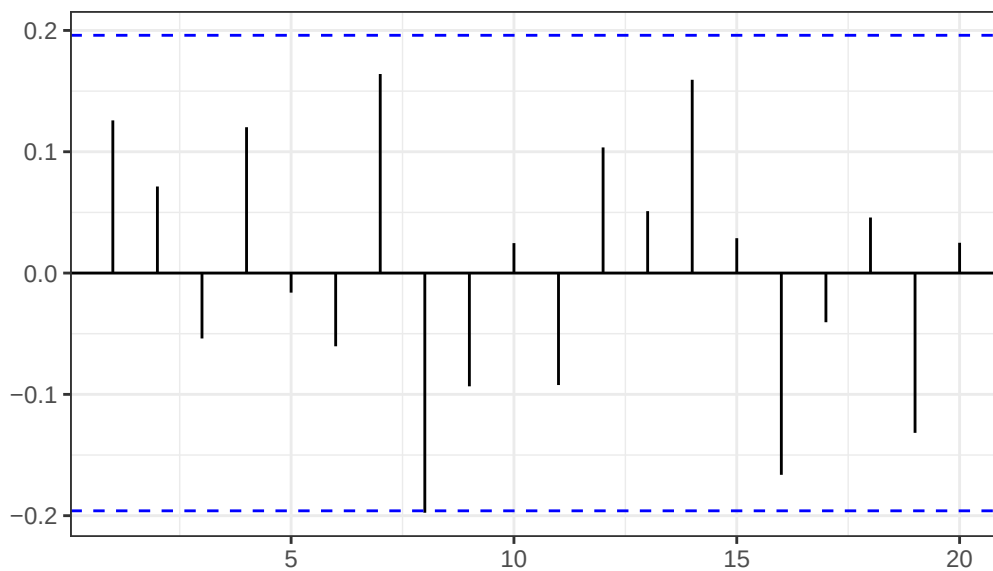
```
# Apply and plot PACF.  
# Correlation only with first lag. Latter lags due to first lag influence.  
PI_train %>%  
  PACF(lag_max = 12, PI_Growth) %>%  
  autoplot() + theme_bw() + labs(x="", y="PACF") +  
  labs(x="", y="", title="PACF Personal Income Growth In California")
```

PACF Personal Income Growth In California



```
# Apply PACF to white noise data. Confirm no correlation.
wn %>% PACF(x) %>% autoplot() + theme_bw() + labs(x="", y="PACF") +
  labs(x="", y="", title="PACF White Noise Process")
```

PACF White Noise Process



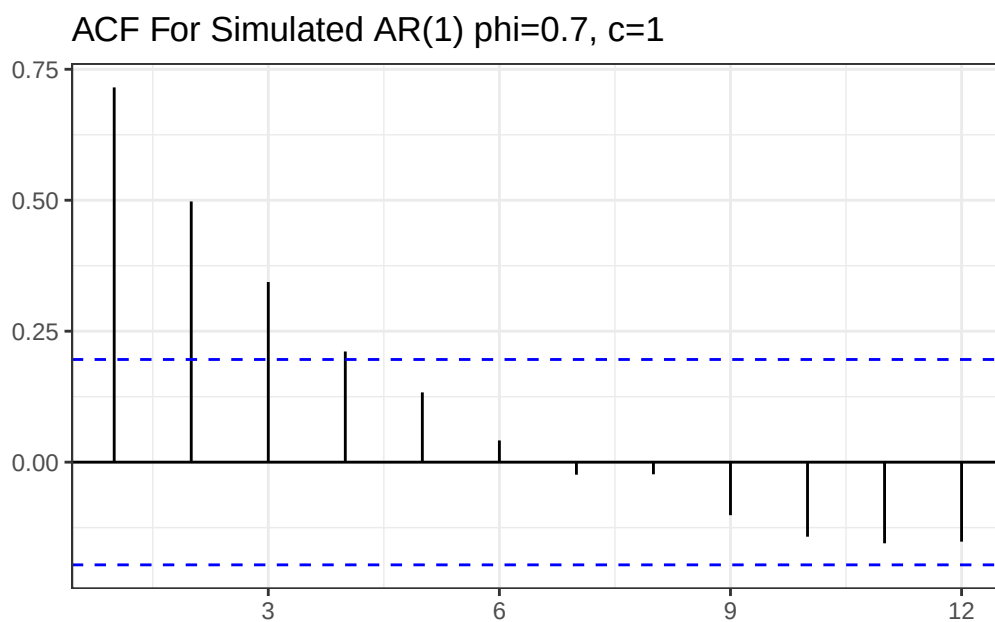
AR(1) Model

Uses only the first lag of the series as independent variable.

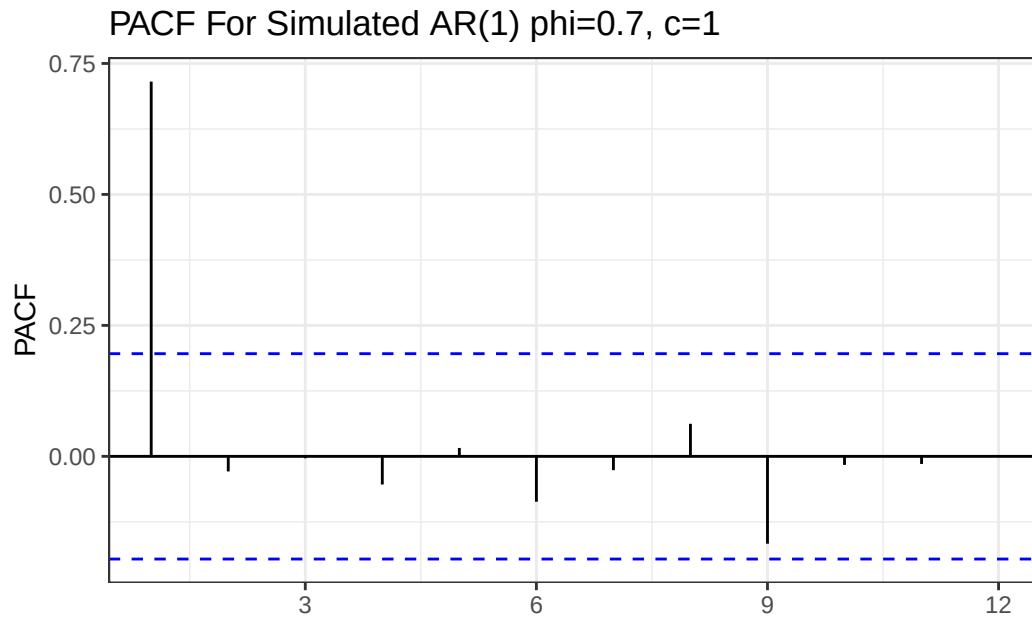
AR(p) uses largest lag.

```
# Simulate data using AR(1) model.  
y<-c(0)  
phi<-0.7  
const<-1  
nrep<-100  
  
for (i in 2:nrep){  
  y[i]=const+phi*y[i-1]+rnorm(1,0,0.5)  
}
```

```
# Apply ACF.  
tsibble(y=y,period=seq(1,length(y)),index=period) %>% ACF(lag_max = 12, y) %>% autoplot()+  
  labs(x="", y="", title="ACF For Simulated AR(1) phi=0.7, c=1")
```



```
# Apply PACF.  
tsibble(y=y,period=seq(1,length(y)),index=period) %>% PACF(lag_max = 12, y) %>% autoplot()+t
```



Modeling and Residuals

Model PI growth using AR(1) model. Include LS model to compare.

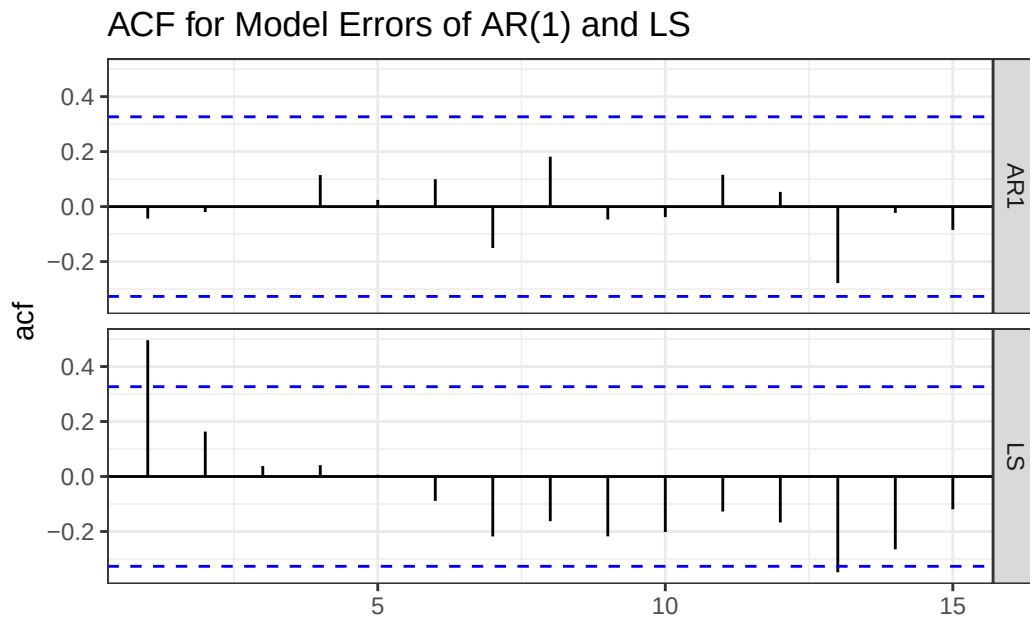
```
PI_fit<-PI_train %>%
  model(AR1 = AR(PI_Growth ~ order(1)),
        LS = TSLM(PI_Growth ~ trend()))
coef(PI_fit)
```

```
# A tibble: 4 x 6
  .model term      estimate std.error statistic  p.value
  <chr>   <chr>      <dbl>    <dbl>    <dbl>    <dbl>
1 AR1    constant    1.80     0.820     2.19 3.54e- 2
2 AR1    ar1          0.702    0.121     5.80 1.56e- 6
3 LS     (Intercept)  9.19     0.799    11.5 2.88e-13
4 LS     trend()     -0.167    0.0376   -4.43 9.31e- 5
```

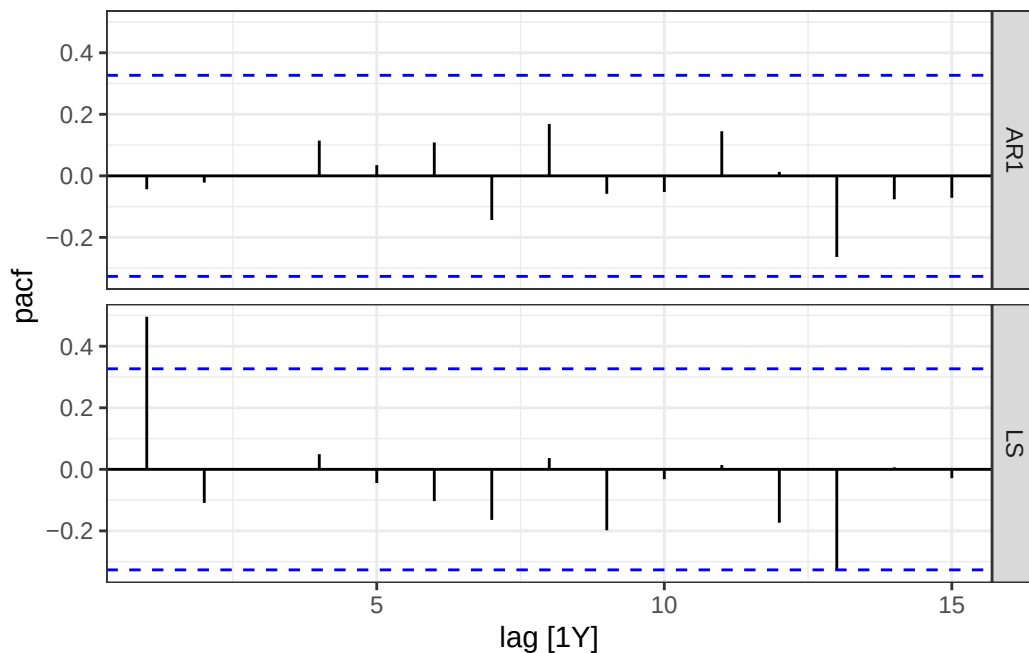
```
# Inspect errors using augment() function.
errors_PI<-augment(PI_fit)

# Plot ACF error.
```

```
# ACF of errors for AR(1) model resemble white noise.
errors_PI %>% select(.resid) %>% ACF(.resid) %>%
  autoplot() + theme_bw() +
  labs(title="ACF for Model Errors of AR(1) and LS",
        x="")
```



```
# Plot PACF error to confirm.
errors_PI %>% select(.resid) %>% PACF(.resid) %>%
  autoplot() + theme_bw()
```



Model Selection

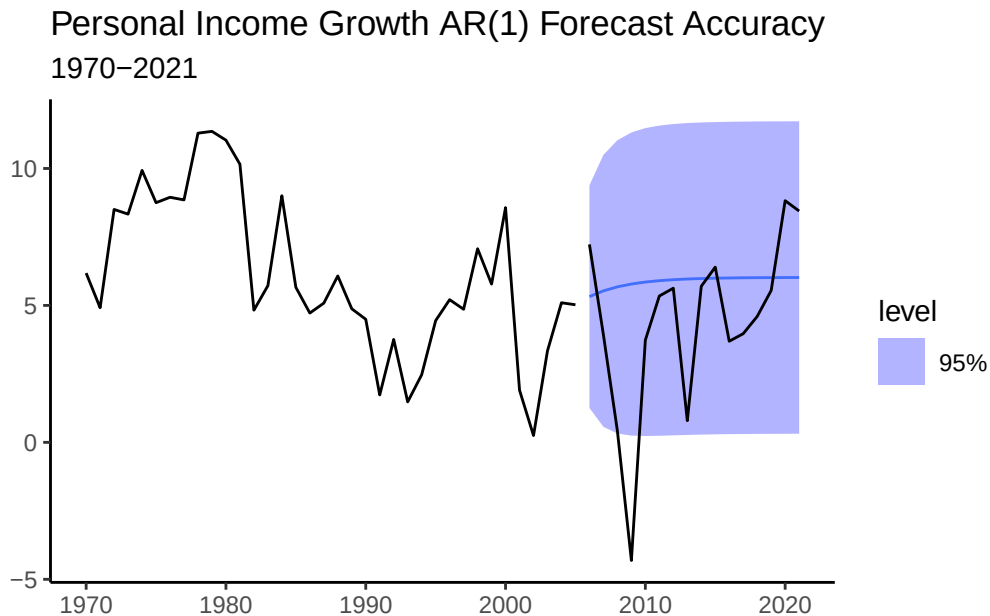
```
# Accuracy measures for AR(1) and LS models.
# AR performs better on all metrics.
glance(PI_fit) %>% arrange(AICc) %>% select(.model:BIC)
```

```
# A tibble: 2 x 5
  .model sigma2  AIC  AICc  BIC
  <chr>   <dbl> <dbl> <dbl> <dbl>
1 AR1     4.30 -20.3 -19.9 -17.1
2 LS      5.51  65.4  66.1  70.1
```

```
# Confirm with test set.
PI_fc<-PI_fit %>% forecast(new_data = PI_test)
PI_fc %>% accuracy(PI_test)
```

```
# A tibble: 2 x 10
  .model .type  ME  RMSE  MAE  MPE  MAPE  MASE  RMSSE  ACF1
  <chr>   <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 AR1   Test  -1.51  3.47  2.45 -116.  157.  NaN  NaN  0.331
2 LS    Test   2.60  4.43  3.87  22.5  107.  NaN  NaN  0.483
```

```
# Plot test set along with AR(1) forecast.
# Blue line indicates mean of predictions.
PI_fc %>% filter(.model=="AR1") %>% autoplot(level=95) + theme_classic() +
  autolayer(PI_train, PI_Growth) +
  autolayer(PI_test, PI_Growth) +
  labs(title="Personal Income Growth AR(1) Forecast Accuracy",
        subtitle="1970-2021", y="", x="")
```



MA(1) Model

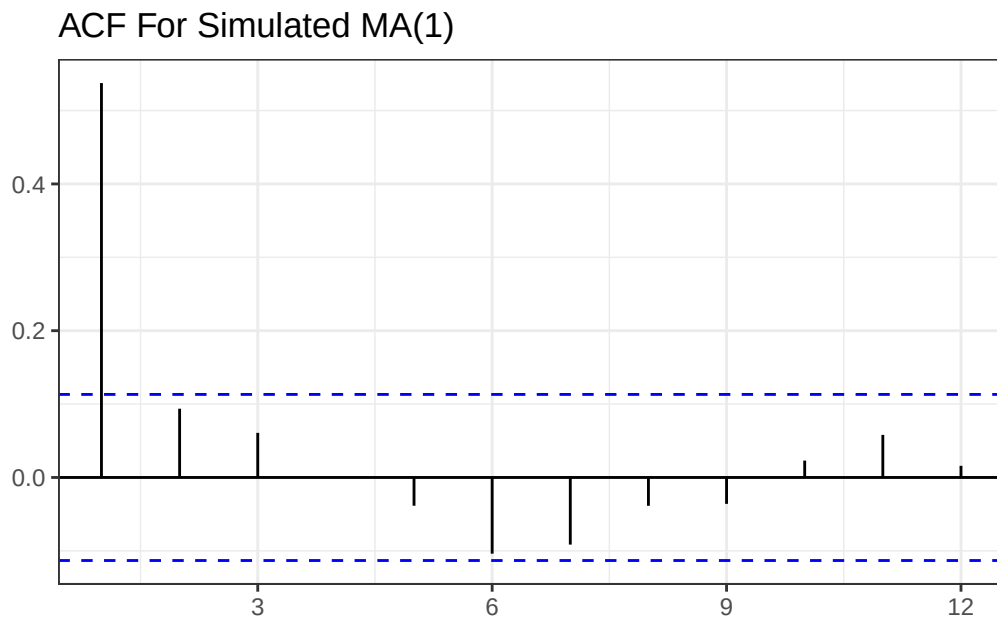
Moving average (MA) process models time series by using past errors (random shocks).

As with the AR model, further error term lags could be included in the MA. An MA process with highest lag of q is called an MA(q) process. The MA process assumes that the current observation of a time series is a linear combination of the weighted sum of past error terms. The error terms capture the unpredictable and random fluctuations in the time series that are not accounted for by the AR model.

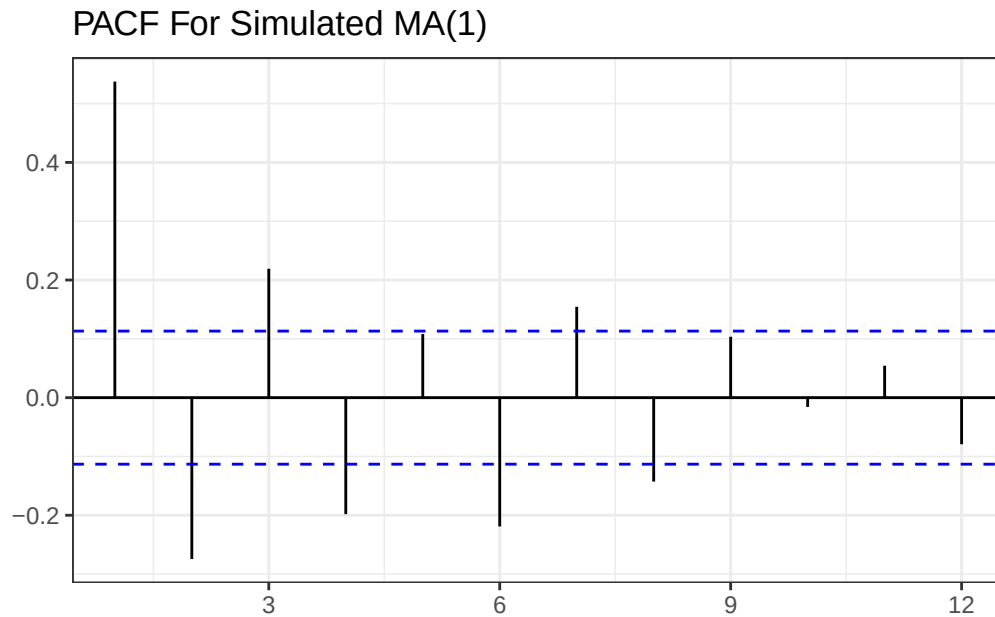
The pattern shown is one significant spike in the first lag of the ACF, and a decaying PACF as lags get larger. This is the opposite of the AR(1) process where we had a decaying ACF and one spike in the PACF.

```
# Simulate MA(1) process and generate ACF/PACF plots.
set.seed(13)
e_t<-rnorm(300,0,0.5)
y_t=2+0.95*dplyr::lag(e_t)+e_t

tsibble(y=y_t,period=seq(1,length(y_t)),index=period) %>% ACF(lag_max = 12, y) %>% autoplot(
labs(x="", y="", title="ACF For Simulated MA(1)")
```



```
tsibble(y=y_t,period=seq(1,length(y_t)),index=period) %>% PACF(lag_max = 12, y) %>% autoplot(
labs(x="", y="", title="PACF For Simulated MA(1)")
```



US Treasury Example

```
# Load data and plot series.
treasury<-read_csv("https://jagelvels.github.io/Data/treasury.csv")
```

Rows: 660 Columns: 2

-- Column specification -----

Delimiter: ","

chr (1): OBS

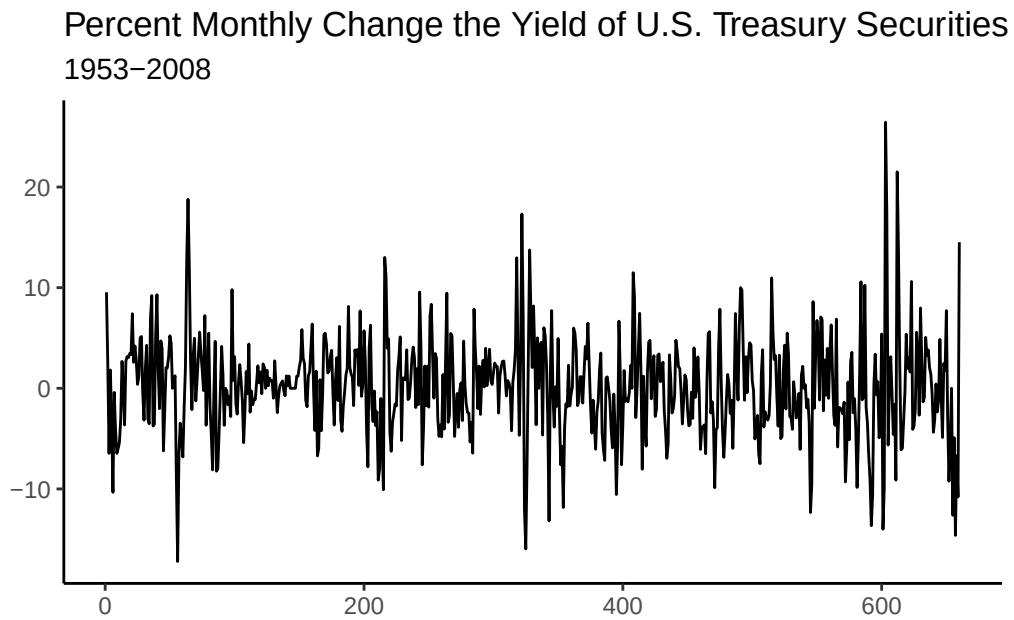
dbl (1): changetreasury

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

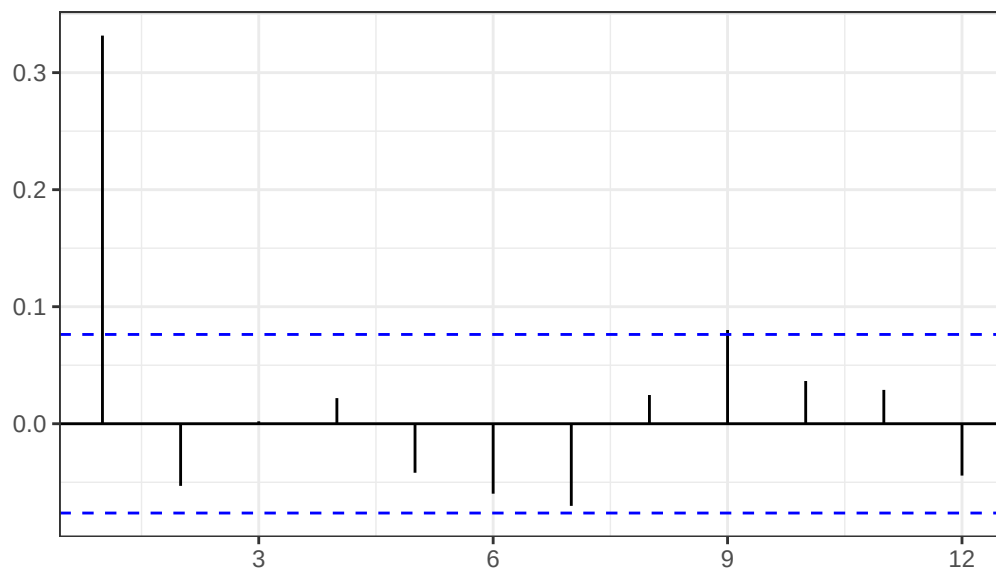
```
treasury %>% mutate(period=1:length(OBS)) -> treasury
as_tsibble(treasury, index = period) %>%
  autoplot() + theme_classic() +
  labs(title="Percent Monthly Change the Yield of U.S. Treasury Securities",
       subtitle="1953-2008",
       x="", y="")
```

Plot variable not specified, automatically selected ``vars = changetreasury``

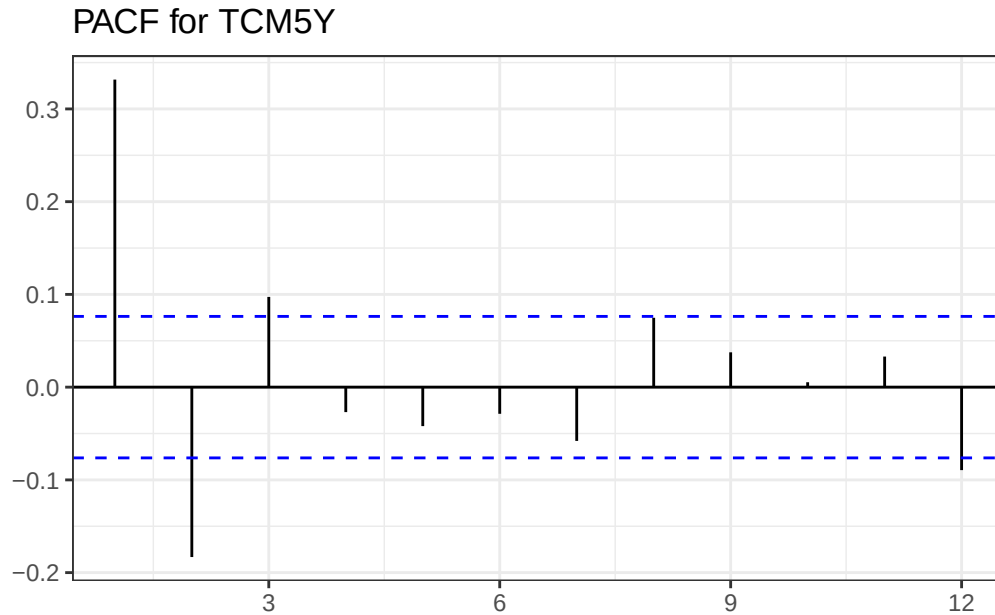


```
#ACF and PACF
as_tsibble(treasury, index = period) %>%
  ACF(lag_max = 12, changetreasury) %>%
  autoplot() + theme_bw() +
  labs(x="", y="", title="ACF for TCM5Y")
```

ACF for TCM5Y



```
as_tsibble(treasury, index = period) %>%  
  PACF(lag_max = 12, changetreasury) %>%  
  autoplot() + theme_bw() +  
  labs(x="", y="", title="PACF for TCM5Y")
```



ARIMA Model

The ARIMA model is based on stationary, AR, and MA processes. ARIMA stands for Autoregressive Integrated Moving Average.

Chilango Restaurant Example

```
# Load data and create test/train sets.
cali<-read_csv("https://jagelves.github.io/Data/CaliforniaAvocado.csv")
```

```
Rows: 205 Columns: 4
```

```
-- Column specification -----
```

```
Delimiter: ","
```

```
chr (1): geography
```

```
dbl (2): average_price, total_volume
```

```
date (1): date
```

```
i Use `spec()` to retrieve the full column specification for this data.
```

```
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
cali %>%
  as_tsibble(key=geography,index=date,regular=T) %>%
  filter_index("2015-01-04"~"2018-12-02") -> cali

cali %>%
  as_tsibble(key=geography,index=date,regular=T) %>%
  filter_index("2015-01-04"~"2018-06-02") -> calits_train

cali %>%
  as_tsibble(key=geography,
             index=date, regular=T) %>%
  filter_index("2018-06-02"~"2018-12-02") -> calits_test
```

```
# Estimate models.
fit <- model(calits_train,ETS=ETS(average_price),
            ARIMA=ARIMA(average_price,approximation = F),
            AR=AR(average_price~order(1)),
            LS=TSLM(average_price~trend()))
```

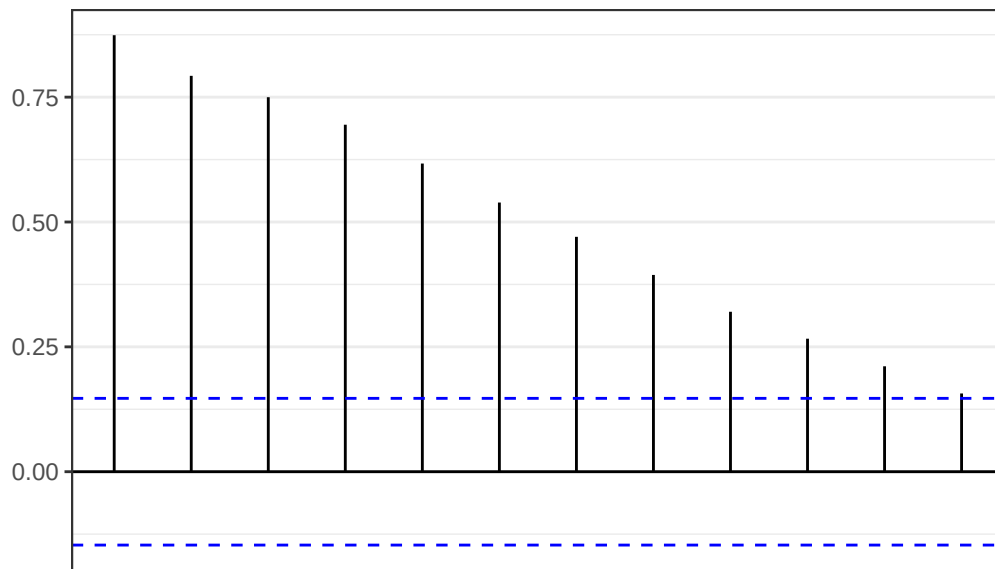
```
# Check coefficients.
fit %>% coef()
```

```
# A tibble: 10 x 7
```

	geography	.model	term	estimate	std.error	statistic	p.value
	<chr>	<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1	California	ETS	alpha	0.646	NA	NA	NA
2	California	ETS	l[0]	1.21	NA	NA	NA
3	California	ARIMA	ar1	0.922	0.110	8.40	1.38e-14
4	California	ARIMA	ar2	-0.736	0.114	-6.49	8.46e-10
5	California	ARIMA	ma1	-1.15	0.0807	-14.2	4.35e-31
6	California	ARIMA	ma2	0.849	0.0966	8.79	1.29e-15
7	California	AR	constant	0.214	0.0605	3.54	5.21e- 4
8	California	AR	ar1	0.874	0.0354	24.7	4.63e-59
9	California	LS	(Intercept)	1.49	0.0391	38.0	8.76e-87
10	California	LS	trend()	0.00222	0.000379	5.87	2.12e- 8

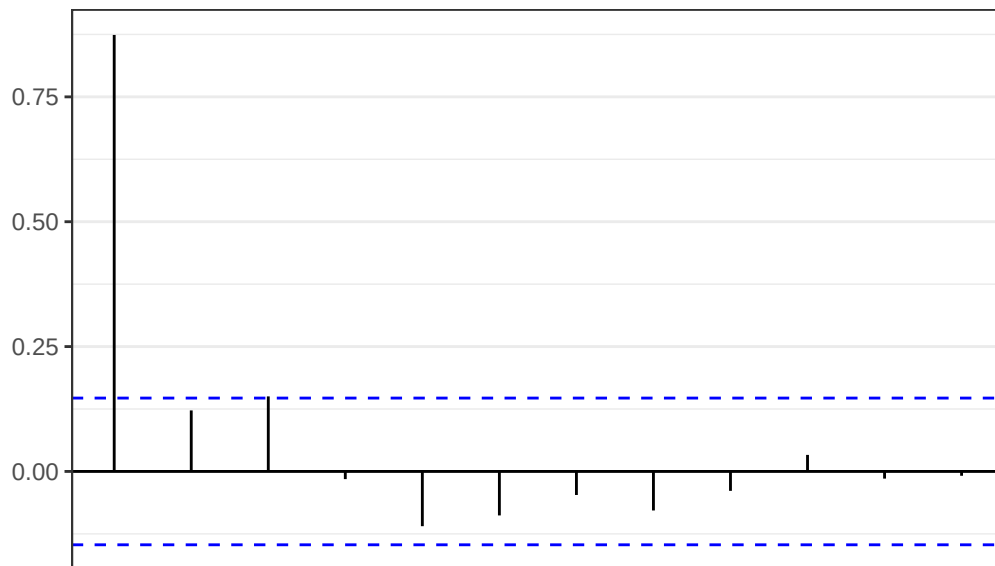
```
calits_train %>% ACF(lag_max=12, average_price) %>%
autoplot()+theme_bw()+
labs(x="", y="", title="ACF For Avocado Average Price")
```

ACF For Avocado Average Price

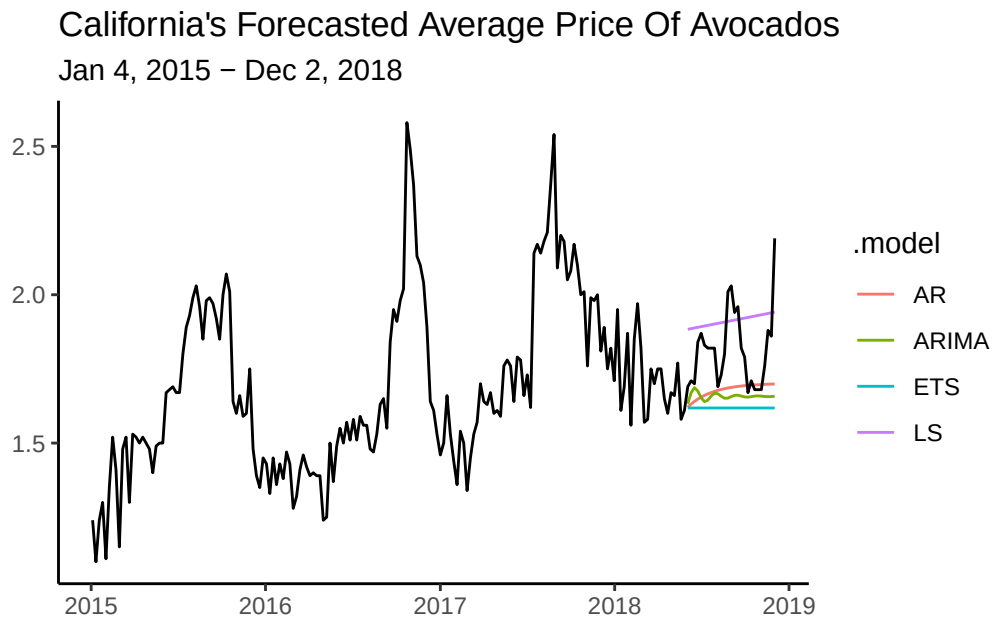


```
calits_train %>% PACF(lag_max=12, average_price) %>%  
autoplot()+theme_bw()+  
labs(x="", y="", title="PACF For Avocado Average Price")
```

PACF For Avocado Average Price



```
# Forecast.
fit %>% forecast(h=27) %>% autoplot(level=NULL) +
  theme_classic() +
  autolayer(cali, average_price) +
  labs(y="", title= "California's Forecasted Average Price Of Avocados",
        subtitle = "Jan 4, 2015 - Dec 2, 2018",
        x="")
```



Ensembles

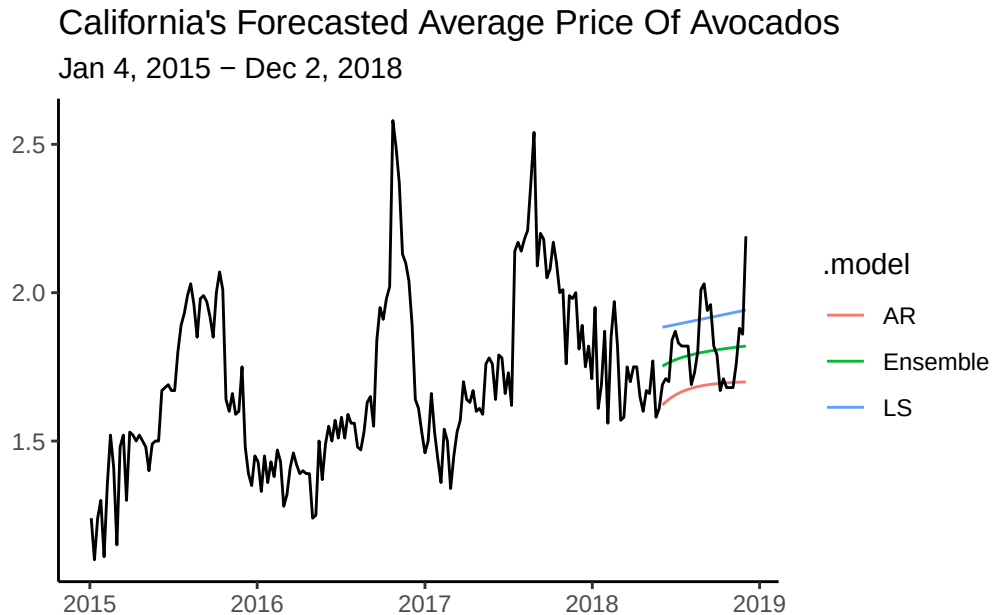
To create a combination of models we can use the fit object containing the mable. Specifically, any model in the fit object can be combined using a linear combination. The code below combines the AR(1) model and the LS model by finding a simple average between both models.

```
fit %>% mutate(Ensemble=0.5*LS+0.5*AR) -> fit2

#Now we can plot the forecasts for the ensemble, the AR1 and the LS model.

fit2 %>% select(AR,LS,Ensemble) %>%
  forecast(h=27) %>% autoplot(level=NULL) +
```

```
autolayer(cali,average_price) + theme_classic() +
labs(y="", title= "California's Forecasted Average Price Of Avocados",
      subtitle = "Jan 4, 2015 - Dec 2, 2018",
      x="")
```



```
fit2 %>% forecast(h=27) %>% accuracy(cali)
```

A tibble: 5 x 11

	.model	geography	.type	ME	RMSE	MAE	MPE	MAPE	MASE	RMSSE	ACF1
	<chr>	<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	AR	California	Test	0.135	0.182	0.141	7.07	7.40	1.40	1.32	0.510
2	ARIMA	California	Test	0.156	0.201	0.156	8.20	8.20	1.55	1.46	0.512
3	ETS	California	Test	0.196	0.233	0.196	10.4	10.4	1.95	1.69	0.502
4	Ensemble	California	Test	0.0186	0.124	0.0966	0.594	5.19	0.961	0.899	0.515
5	LS	California	Test	-0.0983	0.158	0.138	-5.88	7.78	1.37	1.14	0.520

Exercises

Question 1

Recall Question 2 in Module 6, where you are tasked to forecast the energy consumption of a new ice cream machine. Use all the data in the Electricity Bill [Excel] Download Electricity

Bill [Excel]file to estimate an AR1 model. Graph the ACF and PACF and confirm that an AR1 is appropriate. Check residuals from the model and confirm that they resemble white noise. Create a graph of the series and the forecast for four periods ahead.

```
Elec<-read_csv("http://jagelvers.github.io/Data/ElectricityBill.csv")
```

```
Rows: 38 Columns: 6
```

```
-- Column specification -----
```

```
Delimiter: ","
```

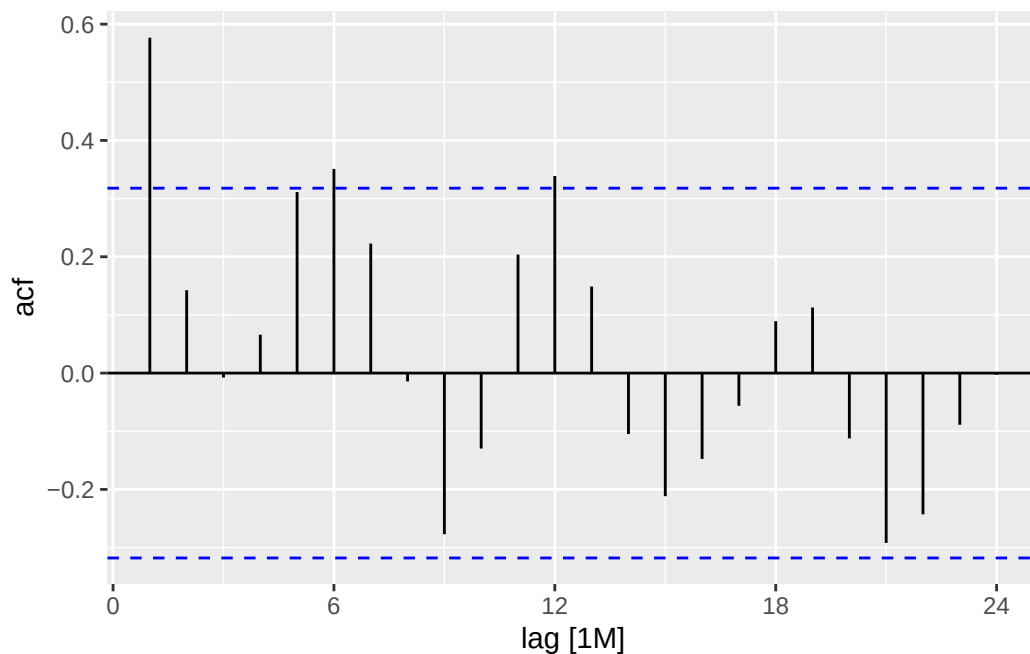
```
dbl (6): Year, Month, Meter Read, Usage (kWh), Avg. Daily Usage, Bill Amount
```

```
i Use `spec()` to retrieve the full column specification for this data.
```

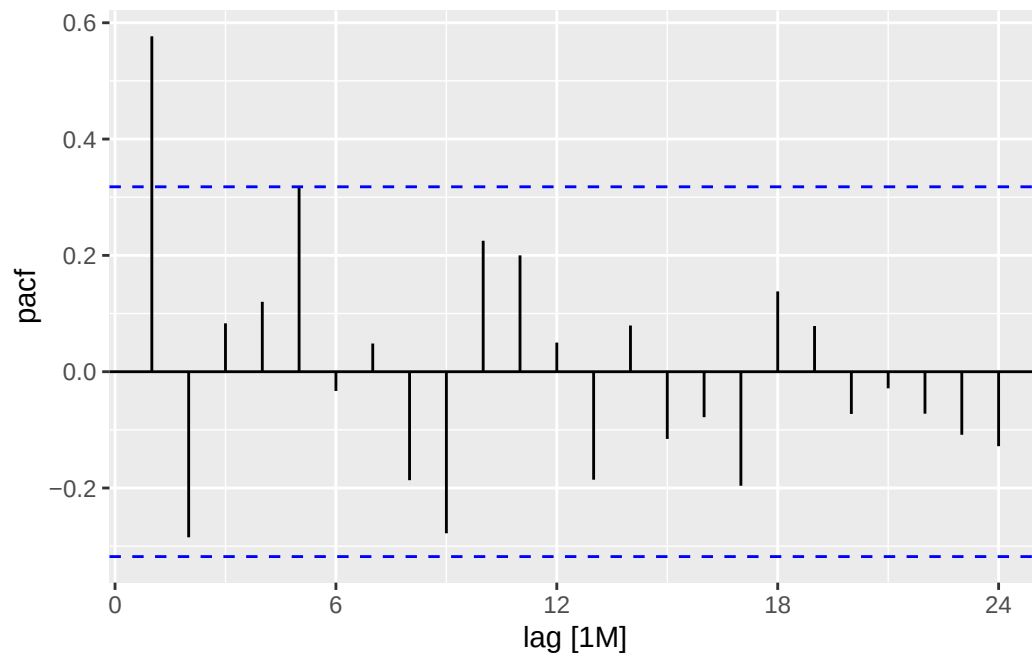
```
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
Elec %>% mutate(Date=yearmonth(paste(Year, Month, sep = "-"))) %>%  
  select(Date, `Bill Amount`) %>%  
  as_tsibble(index=Date)-> Elec_ts
```

```
Elec_ts %>% ACF(`Bill Amount`, lag_max = 24) %>% autoplot()
```

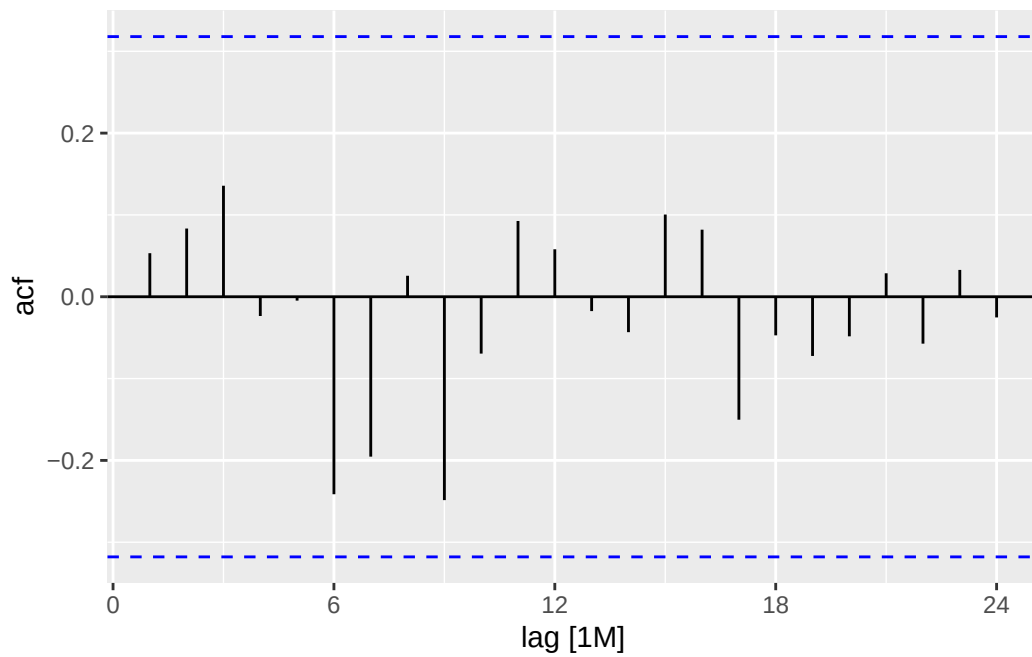


```
Elec_ts %>% PACF(`Bill Amount`,lag_max = 24) %>% autoplot()
```

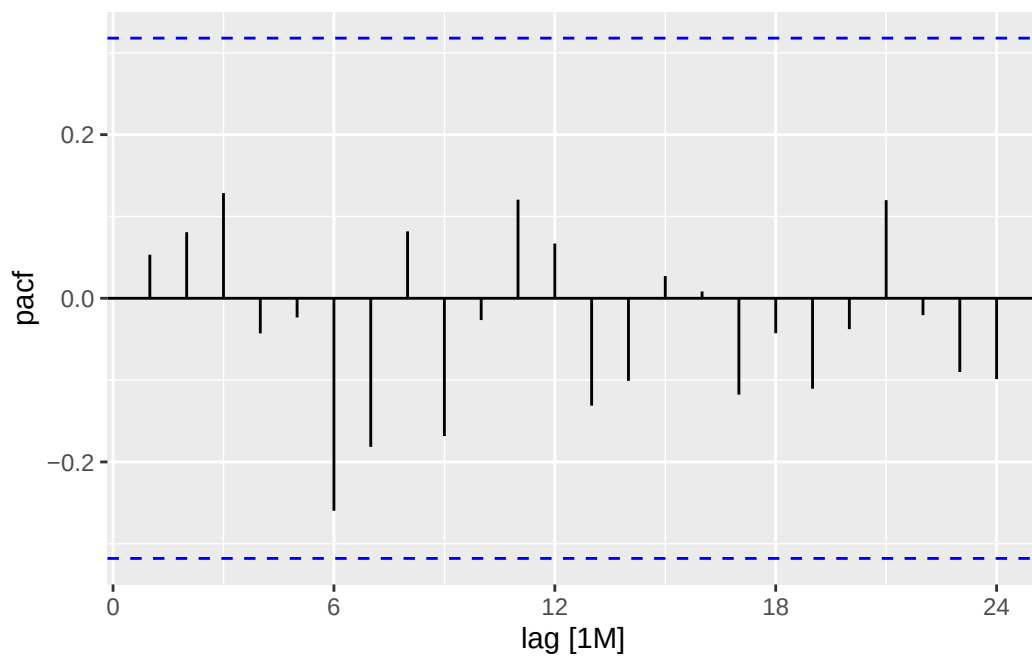


```
Elec_ts %>% model(AR1=ARIMA(`Bill Amount`~pdq(1,0,0))) -> fit
```

```
fit %>% augment() %>% ACF(.resid,lag_max = 24) %>% autoplot()
```



```
fit %>% augment() %>% PACF(.resid, lag_max = 24) %>% autoplot()
```



Question 2

Recall Question 1 in Module 6, where you are tasked to forecast Tesla's energy deployment. Use the entire data in the Electricity Bill Excel file to estimate an ensemble of the TSLM model with linear trend and seasonality, and a TSLM model with quadratic trend and seasonality. Let the weight of the linear model be 30% and that of the quadratic model 70%. Graph the forecast for the next four periods of the three models.

```
library(ggthemes)
```

Warning: package 'ggthemes' was built under R version 4.4.2

```
storage<-read_csv("http://jagelves.github.io/Data/teslaE.csv")
```

Rows: 15 Columns: 2

-- Column specification -----

Delimiter: ","

chr (1): Date

dbl (1): EnergyStorage

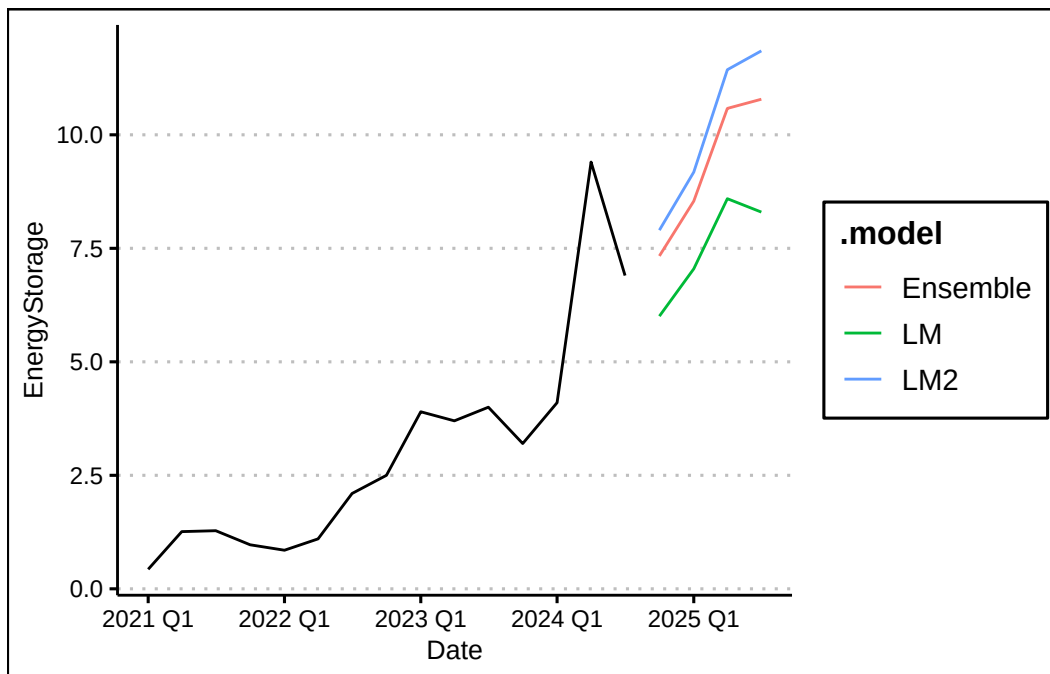
i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
storage %>%  
  mutate(Date=yearquarter(Date)) %>%  
  as_tsibble(index=Date) -> storage_ts
```

```
storage_ts %>%  
  model(LM=TSLM(EnergyStorage~trend()+season()),  
        LM2=TSLM(EnergyStorage~trend()+I(trend()^2)+season())) ->fit  
  
fit %>% mutate(Ensemble=0.3*LM+0.7*LM2) -> fit2
```

```
fit2 %>% forecast(h=4) %>%  
  autoplot(level=NULL) +  
  autolayer(storage_ts,EnergyStorage) +  
  theme_clean()
```



Question 3

Using the monthly inflation data from the Inflation Excel file, forecast the inflation rate for the next four months. First, use the `ARIMA()` function to let the model automatically select the best-fitting ARIMA model for the inflation data while incorporating the interest rate into the model with the command `ARIMA(Inflation~InterestRate)`. Develop two scenarios: the Federal Reserve holds the interest rate steady at 5.33% over the next four months, and the Fed follows a decreasing schedule of 5.25%, 5.00%, 4.75%, and 4.50%. Identify the month the Fed reaches its target of 2% inflation in each scenario and determine which schedule achieves this target faster. Create a graph that shows each scenario and the target inflation rate.

```
library(ggthemes)
data<-read_csv("https://jagelves.github.io/Data/Inflation.csv")
```

```
Rows: 54 Columns: 5
```

```
-- Column specification -----
```

```
Delimiter: ","
```

```
chr (1): Date
```

```
dbl (4): Inflation, InterestRate, S&Preturns, Unemployment
```

```
i Use `spec()` to retrieve the full column specification for this data.
```

```
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
data %>% mutate(Date=yearmonth(mdy(Date))) %>%
  as_tsibble(index=Date) -> data_ts
```

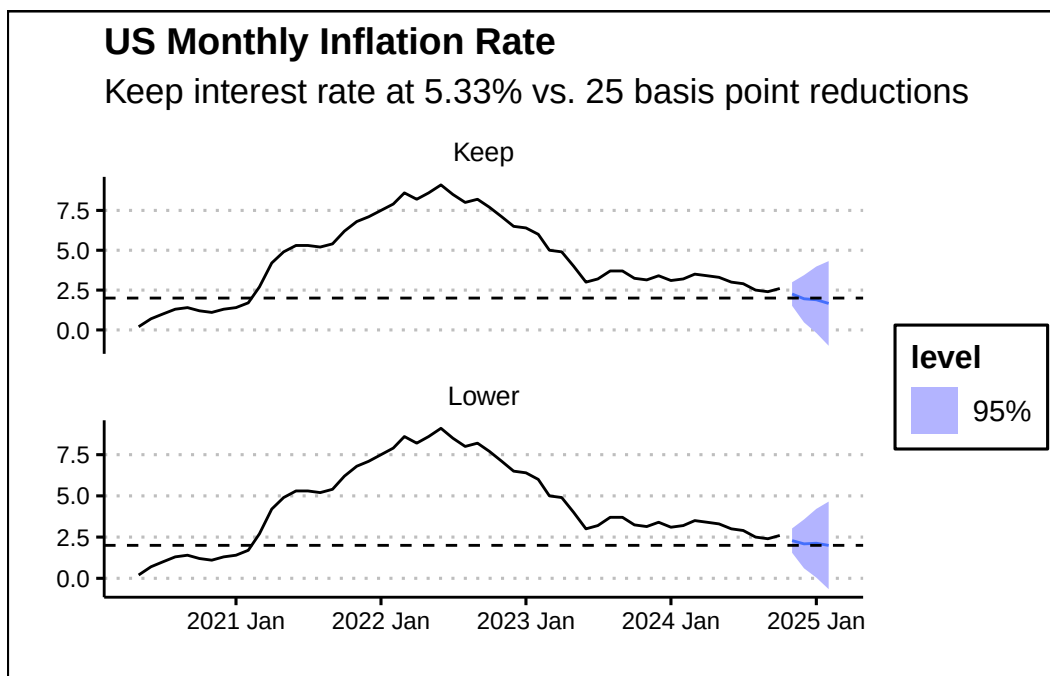
```
data_ts %>%
  model(ARIMA=ARIMA(Inflation~InterestRate)) -> fit2
```

```
Int <- scenarios(
  Keep = new_data(data_ts, 4) %>%
    mutate(InterestRate=rep(5.33,4)),
  Lower = new_data(data_ts, 4) %>%
    mutate(InterestRate=c(5.25,5.00,4.75,4.5)),
  names_to = "Scenario")
```

```
forecast(fit2,new_data=Int)
```

```
# A fable: 8 x 6 [1M]
# Key:      Scenario, .model [2]
#   Scenario .model   Date
#   <chr>    <chr>    <mth>
1 Keep      ARIMA    2024 Nov
2 Keep      ARIMA    2024 Dec
3 Keep      ARIMA    2025 Jan
4 Keep      ARIMA    2025 Feb
5 Lower     ARIMA    2024 Nov
6 Lower     ARIMA    2024 Dec
7 Lower     ARIMA    2025 Jan
8 Lower     ARIMA    2025 Feb
# i 3 more variables: Inflation <dist>, .mean <dbl>, InterestRate <dbl>
```

```
forecast(fit2,new_data=Int) %>% autoplot(level=95) +
  autolayer(data_ts,Inflation) +
  theme_clean() +
  geom_hline(yintercept=2, lty=2) +
  labs(title="US Monthly Inflation Rate",
       subtitle="Keep interest rate at 5.33% vs. 25 basis point reductions",
       x="",y="")
```



Question 4

Rising U.S. household debt is concerning because it increases financial strain on families, reduces disposable income, and heightens vulnerability to interest rate hikes. It raises the risk of defaults and bankruptcies, potentially destabilizing financial institutions and the broader economy. Additionally, higher debt levels can slow consumer spending and economic growth, while also creating risks in the housing market, making the economy more fragile during downturns. Use the data in the U.S. Debt Excel file to forecast the US total debt of households using ARIMA. Start by graphing the ACF and PACF, then suggest two ARIMA models and let the algorithm choose the third. Calculate the BIC for the three models and select the model with the lowest BIC. Forecast the series into the future. By what date does the model predict the US Household debt will reach 20 trillion dollars?

```
data<-read_csv("https://jagelves.github.io/Data/USDebt.csv")
```

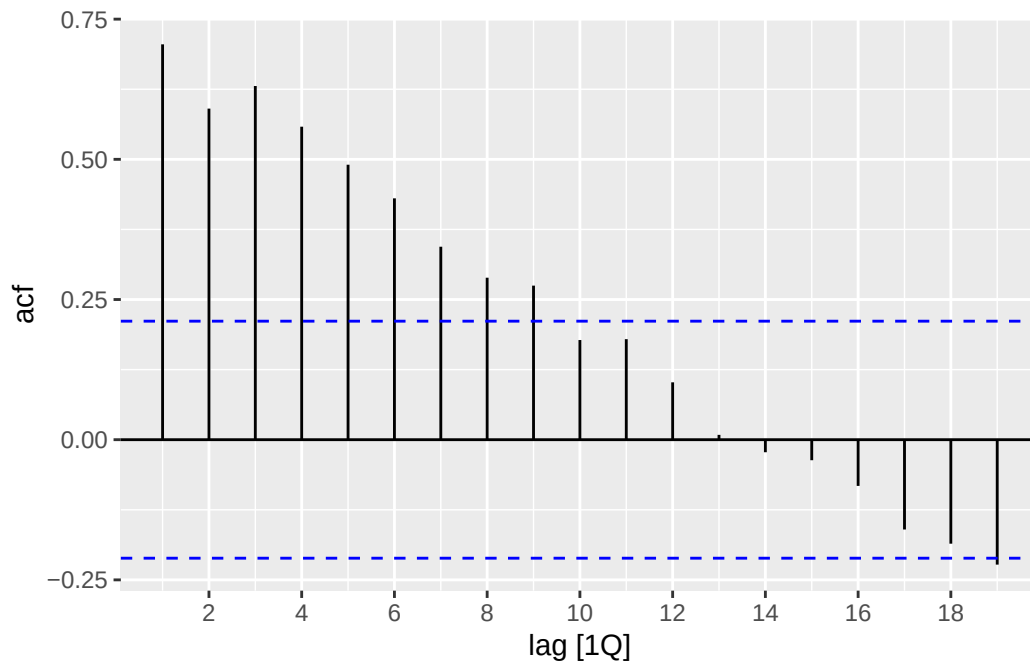
```
Rows: 86 Columns: 9
-- Column specification -----
Delimiter: ","
chr (1): Quarter
dbl (8): Mortgage, HERevolving, AutoLoan, CreditCard, StudentLoan, Other, To...

i Use `spec()` to retrieve the full column specification for this data.
```

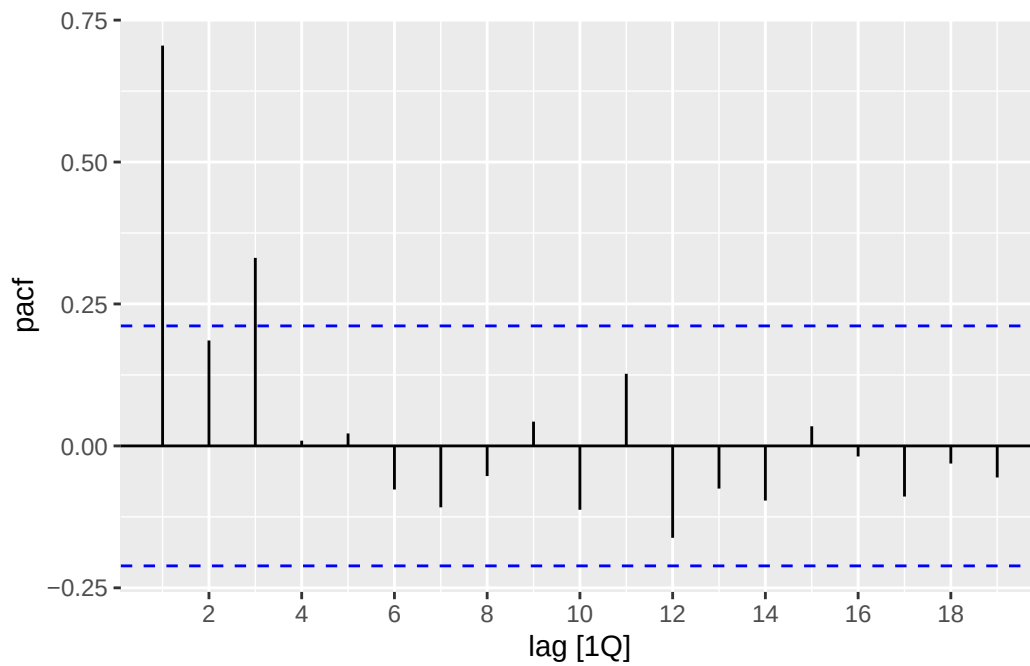
i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
data %>% mutate(Quarter=yearquarter(Quarter)) %>%  
  as_tsibble(index=Quarter) -> data_ts
```

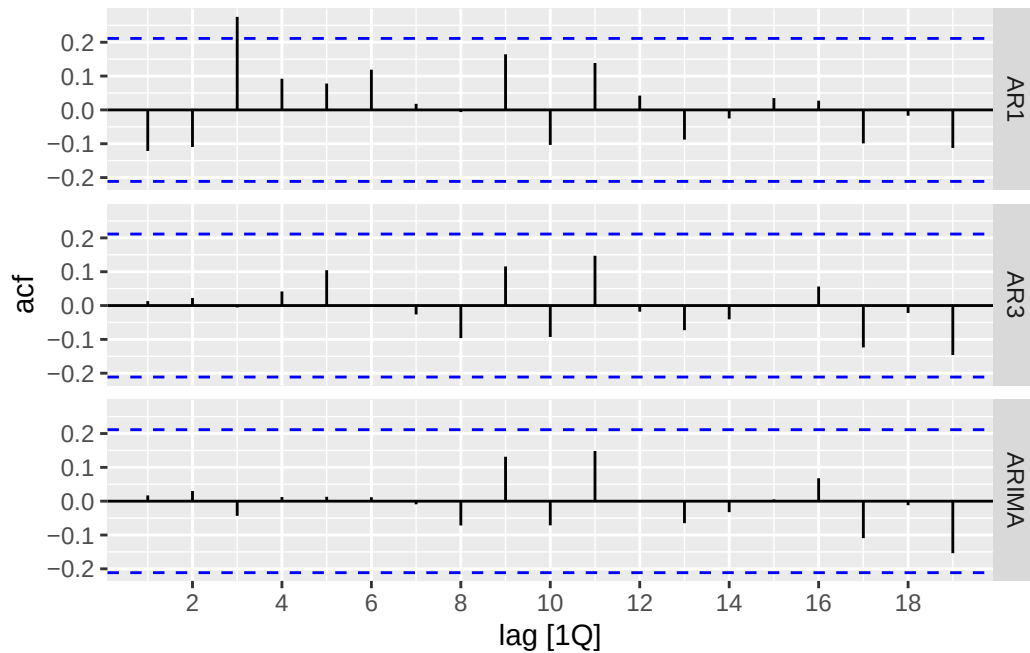
```
data_ts %>% ACF(diff(Total)) %>% autoplot()
```



```
data_ts %>% PACF(diff(Total)) %>% autoplot()
```



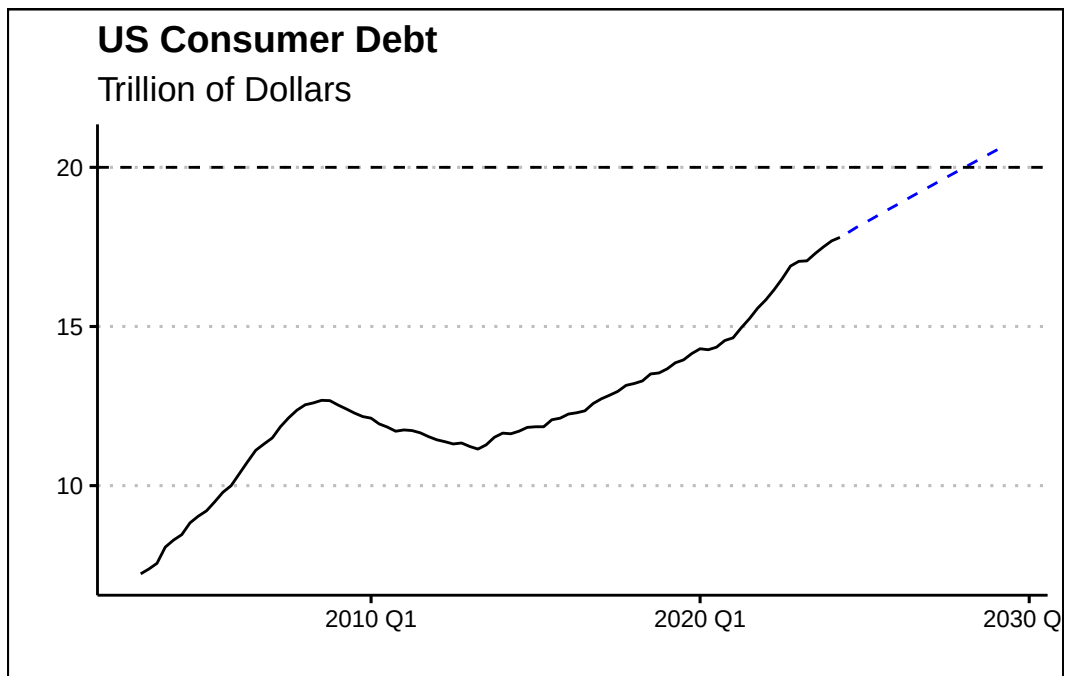
```
data_ts %>%  
  model(AR3=ARIMA(Total~pdq(3,1,0)),  
        AR1=ARIMA(Total~pdq(1,1,0)),  
        ARIMA=ARIMA(Total)) -> fit  
  
#Now we can observe the residuals:  
  
fit %>% augment() %>% ACF(.resid) %>% autoplot()
```



```
fit %>% glance() %>% select(.model,BIC)
```

```
# A tibble: 3 x 2
  .model    BIC
  <chr>   <dbl>
1 AR3    -131.
2 AR1    -125.
3 ARIMA  -128.
```

```
fit %>% forecast(h=20) %>%
  as_tsibble() %>% filter(.model=="AR3") %>%
  autoplot(.mean, col="blue", lty=2) +
  autolayer(data_ts,Total) + theme_clean() +
  labs(title="US Consumer Debt",
        subtitle="Trillion of Dollars",
        x="",y="") +
  geom_hline(yintercept=20,lty=2)
```



[1] 4

The `echo: false` option disables the printing of code (only output is displayed).